

Cálculo diferencial y optimización

Módulo 2 · Matemáticas y Fundamentos Técnicos de IA

Gradientes, backpropagation, Adam, learning rate y los problemas del gradiente: por que el entrenamiento converge o diverge.

1. Introducción · 2. Qué es · 3. Cómo funciona · 4. Ejemplos reales
5. Errores comunes · 6. Aplicación práctica · 7. Relación con módulos · 8. Resumen ejecutivo

Guía de estudio detallada · +2.000 palabras

Cálculo diferencial y optimización en IA

El entrenamiento de cualquier red neuronal, desde el clasificador más simple hasta GPT-4, es un problema de optimización: encontrar los millones o miles de millones de parámetros que minimizan una función de pérdida que mide cuánto se equivoca el modelo. El cálculo diferencial es la herramienta matemática que hace posible resolver ese problema de forma eficiente: los gradientes indican en qué dirección y con qué intensidad ajustar cada parámetro para reducir el error.

Sin el cálculo diferencial no habría backpropagation. Sin backpropagation no habría entrenamiento de redes profundas. Sin entrenamiento de redes profundas no habría deep learning ni LLMs. Entender el cálculo diferencial aplicado a IA no es una curiosidad académica: es entender por qué el entrenamiento de los modelos converge o diverge, por qué ciertas tasas de aprendizaje funcionan y otras no, y por qué técnicas como los residual connections o el gradient clipping existen.

Derivadas, gradientes y la geometría de la optimización

Derivadas: la tasa de cambio instantánea

La derivada de una función f en un punto x mide cuánto cambia f cuando x cambia infinitesimalmente: $f'(x) = \lim_{h \rightarrow 0} [f(x+h) - f(x)] / h$. En el contexto de redes neuronales, si f es la función de pérdida y x es un peso de la red, la derivada $\partial L / \partial w$ mide cuánto aumenta la pérdida si aumentamos w ligeramente. Si esa derivada es positiva, para reducir la pérdida debemos reducir w . Si es negativa, debemos aumentar w .

Gradiente: la derivada en muchas dimensiones

El gradiente de una función $L(w_1, w_2, \dots, w_n)$ respecto a todos sus parámetros es el vector de todas las derivadas parciales: $\nabla L = [\partial L / \partial w_1, \partial L / \partial w_2, \dots, \partial L / \partial w_n]$. El gradiente tiene una propiedad geométrica fundamental: apunta en la dirección de máximo crecimiento de L . Si queremos minimizar L , debemos movernos en la dirección opuesta al gradiente: el descenso de gradiente.

Backpropagation: el gradiente en redes profundas

Calcular el gradiente de la pérdida respecto a todos los pesos de una red profunda requiere aplicar la regla de la cadena del cálculo de forma eficiente. Si la red tiene capas $f_1, f_2, \dots, f_n$ en secuencia, y la pérdida L depende del output de f_n , que depende del output de f_{n-1} , etc., la regla de la cadena dice: $\partial L / \partial w_n = (\partial L / \partial \text{output}_n) \times (\partial \text{output}_n / \partial \text{output}_{n-1}) \times \dots \times (\partial \text{output}_2 / \partial w_1)$. Backpropagation calcula este producto de forma eficiente propagando los gradientes desde la última capa hacia las primeras, reutilizando cálculos intermedios en lugar de recalcularlos.

La eficiencia de backpropagation es lo que hace entrenable las redes profundas. Sin él, calcular el gradiente para una red de 100 capas requeriría tantas evaluaciones de la función como

parámetros tiene la red (millones o miles de millones). Backpropagation lo calcula con una sola pasada hacia atrás, con coste proporcional al número de operaciones en el forward pass.

SECCIÓN 3 · CÓMO FUNCIONA

Descenso de gradiente y sus variantes

Batch Gradient Descent, SGD y Mini-batch

El descenso de gradiente en batch calcula el gradiente sobre todo el conjunto de entrenamiento antes de actualizar los pesos: $w \leftarrow w - \alpha \cdot \nabla L(w)$, donde α es la tasa de aprendizaje. Es estable pero muy lento para datasets grandes (imagina calcular el gradiente sobre 1.000 millones de ejemplos antes de cada actualización). El descenso de gradiente estocástico (SGD) actualiza los pesos después de cada ejemplo: más ruidoso pero mucho más rápido. El mini-batch SGD (el estándar en práctica) actualiza después de pequeños lotes (batches) de 32-512 ejemplos: equilibra velocidad y estabilidad.

Adam: el optimizador estándar de la era del deep learning

Adam (Adaptive Moment Estimation, Kingma y Ba, 2014) es el optimizador más usado en el entrenamiento de LLMs y redes profundas. Mantiene dos estimaciones para cada parámetro: m_t (media de los gradientes pasados, el "momento") y v_t (media de los cuadrados de los gradientes, la "varianza adaptativa"). La actualización es: $w \leftarrow w - \alpha \cdot m_t / (\sqrt{v_t} + \epsilon)$. Esto adapta la tasa de aprendizaje de forma efectiva por parámetro: parámetros con gradientes grandes reciben actualizaciones más pequeñas, parámetros con gradientes pequeños reciben actualizaciones más grandes. El resultado es convergencia más rápida y más estable que SGD puro.

Learning rate: el hiperparámetro más crítico

La tasa de aprendizaje α determina el tamaño de los pasos en el descenso de gradiente. Si α es demasiado grande, los pasos son tan grandes que el optimizador salta sobre los mínimos y el entrenamiento diverge. Si α es demasiado pequeña, la convergencia es extremadamente lenta. Los schedulers de learning rate ajustan α a lo largo del entrenamiento: warmup lineal (empezar con lr pequeño y subirlo gradualmente, evitando inestabilidad al inicio) seguido de cosine decay (reducir lr gradualmente siguiendo una curva coseno, permitiendo refinamiento fino al final del entrenamiento).

Problemas del gradiente en redes profundas

El problema del gradiente desvaneciente (vanishing gradient) ocurre en redes muy profundas: al propagar los gradientes hacia atrás con la regla de la cadena, cada capa multiplica el gradiente por su derivada. Si esas derivadas son menores de 1 (lo que ocurre con funciones de activación saturadas como sigmoid o tanh), el gradiente se hace exponencialmente pequeño en las capas iniciales, que prácticamente no aprenden. Las residual connections (conexiones que saltan capas) de ResNet y Transformer resuelven este problema permitiendo al gradiente fluir directamente sin pasar por las multiplicaciones saturadas.

SECCIÓN 4 · EJEMPLOS REALES

- Fine-tuning de LLM divergiendo: un equipo intenta hacer fine-tuning de Llama 3 8B con una tasa de aprendizaje de $5e-4$. La pérdida sube en lugar de bajar. Diagnóstico: lr demasiado alta para fine-tuning de LLM. Solución: reducir a $2e-4$ con warmup de 100 pasos y cosine decay. El entrenamiento converge en 3 épocas.
- AdamW en GPT-4: la documentación técnica de OpenAI menciona el uso de AdamW (Adam con weight decay explícito) para el entrenamiento de GPT-4, con un learning rate schedule de cosine annealing y warmup. Los hiperparámetros exactos son propietarios, pero el patrón es estándar en la literatura.
- Gradient clipping en LLMs: la práctica estándar en el entrenamiento de LLMs es aplicar gradient clipping con $\text{max_norm}=1.0$. Esto limita la norma del vector de gradiente a 1 antes de la actualización. Previene que spikes ocasionales en los gradientes (causados por ejemplos de entrenamiento atípicos) produzcan actualizaciones desestabilizadoras.
- Learning rate finder: la librería PyTorch Lightning implementa un learning rate finder que entrena brevemente con una gama de tasas de aprendizaje y encuentra la que produce la caída de pérdida más rápida. Es una forma sistemática de calibrar este hiperparámetro crítico sin hacer ensayo-error manual.

SECCIÓN 5 · ERRORES Y MALENTENDIDOS COMUNES

Error 1: "Un lr más alto siempre entrena más rápido." Un lr demasiado alto produce divergencia (la pérdida sube en lugar de bajar). El óptimo es específico del modelo, el optimizador, el tamaño del batch y los datos. Hay que calibrarlo sistemáticamente.

Error 2: "Adam siempre es mejor que SGD." No para todos los casos. Para algunas tareas de visión artificial y para el entrenamiento desde cero de modelos grandes, SGD con momentum y lr schedule cuidadoso puede producir mejores resultados de generalización que Adam. En fine-tuning de LLMs, AdamW es el estándar.

Error 3: "El gradiente desvaneciente ya no existe con ReLU." ReLU mitiga el problema porque su derivada es 1 para valores positivos (no satura en esa dirección), pero puede producir neuronas "muertas" (siempre activas con derivada 0 para valores negativos). Las conexiones residuales son la solución más robusta para redes muy profundas.

SECCIÓN 6 · APLICACIÓN PRÁCTICA

El conocimiento del cálculo diferencial en optimización tiene tres aplicaciones prácticas directas en proyectos empresariales de IA. Primera: diagnóstico de problemas de entrenamiento. Si una curva de pérdida sube o se vuelve inestable, el diagnóstico es casi siempre un problema de tasa de aprendizaje o de gradientes. Saber esto permite intervenir con soluciones específicas en lugar de prueba y error ciega.

Segunda: elección del optimizador y sus hiperparámetros. Para fine-tuning de LLMs: AdamW con lr entre $1e-5$ y $3e-4$, warmup lineal de 100-500 pasos, cosine decay. Para entrenamiento de redes de visión desde cero: SGD con momentum 0.9 y lr schedule cíclico. Estos defaults informados ahorran decenas de horas de experimentos fallidos.

Tercera: evaluación de la convergencia. Monitorear la curva de pérdida de entrenamiento y validación durante el fine-tuning con herramientas como Weights & Biases o MLflow permite

detectar early overfitting (divergencia entre train y val loss) y aplicar early stopping antes de degradar la generalización del modelo.

SECCIÓN 7 · RELACIÓN CON OTROS MÓDULOS

- M1-T3 (IA Conexionista): el backpropagation y el descenso de gradiente son el mecanismo de aprendizaje del conexionismo.
- M4 (Deep Learning): todas las técnicas de entrenamiento de redes profundas (batch normalization, residual connections, data augmentation) tienen su fundamento en los problemas del gradiente abordados aquí.
- M8 (LLMOps / Fine-tuning): LoRA, QLoRA y el fine-tuning eficiente son aplicaciones directas de la optimización con gradientes bajo restricciones de bajo rango.

SECCIÓN 8 · RESUMEN EJECUTIVO

El cálculo diferencial es el motor del aprendizaje en IA: los gradientes indican cómo ajustar los parámetros para reducir el error. Backpropagation aplica la regla de la cadena de forma eficiente para calcular gradientes en redes profundas. Adam es el optimizador estándar: adapta la tasa de aprendizaje por parámetro usando momentos de primer y segundo orden. La tasa de aprendizaje es el hiperparámetro más crítico: demasiado alta produce divergencia, demasiado baja produce convergencia lenta. Gradient clipping, residual connections y warmup son las principales salvaguardas contra los problemas del gradiente en modelos modernos.